



INTRODUCTION TO ROUTING

Full Stack Development Subject

SESSION AGENDA

Full Stack Development Subject

- Quick recap of yesterday
- Nested Routing
- URL Params
- Protected Routes
- 404 + useNavigate

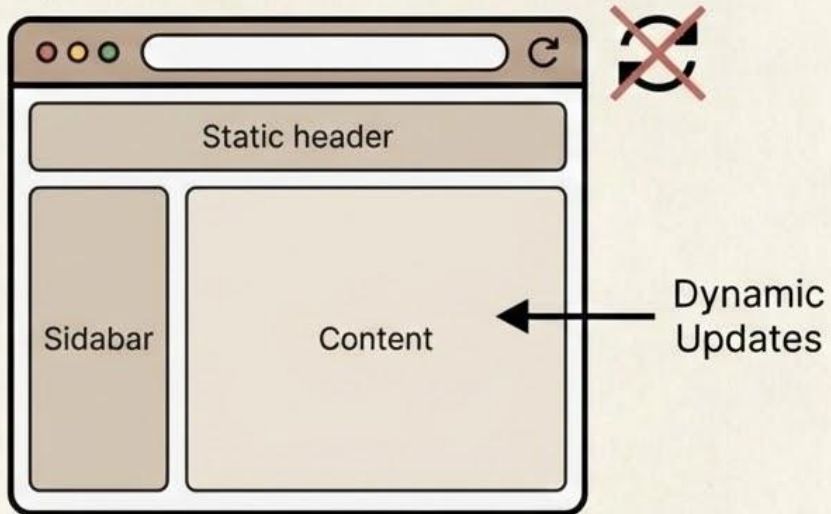
SESSION AGENDA

Full Stack Development Subject

- Quick recap of yesterday
- Nested Routing
- URL Params

SINGLE PAGE APPLICATION (SPA)

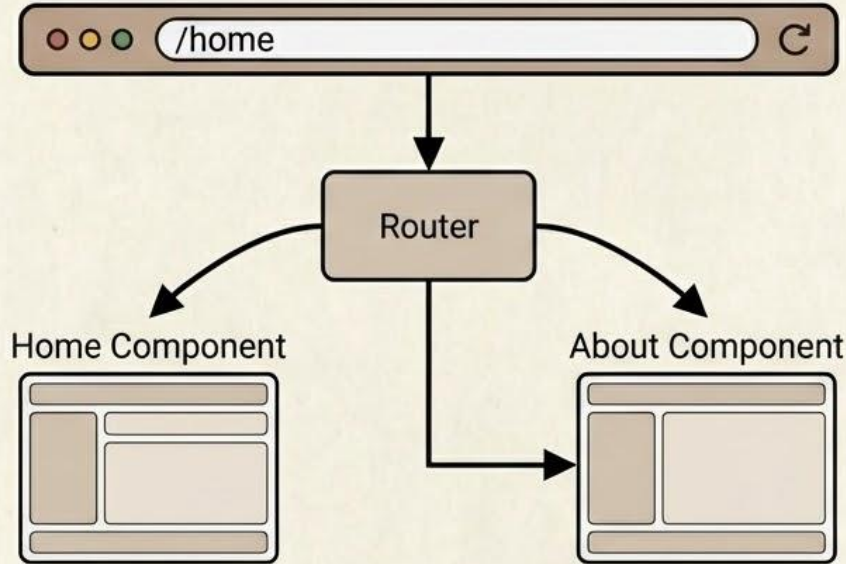
Full Stack Development Subject



- Loads a single HTML page.
- Dynamically updates content without full page reloads.
- Provides a smoother, faster user experience.

ROUTING DEFINITION

Full Stack Development Subject

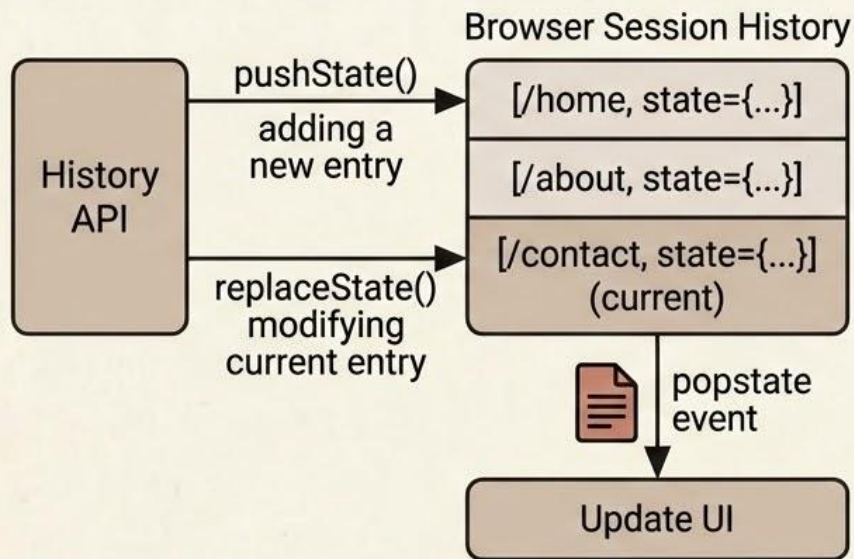


Maps URLs to specific components or views.

- The mechanism that manages navigation within an application.
- Connects URL paths to different parts of the UI.
- Allows users to move between views without a server request.

THE HISTORY API

Full Stack Development Subject



Manipulates browser session history
without page reloads.

- Allows programmatic control of the browser's history stack.
- Key methods: `pushState()`, `replaceState()`.
- Key event: `popstate` for back/forward navigation.
- Essential for implementing client-side routing in SPAs.

INSTALLING REACT ROUTER

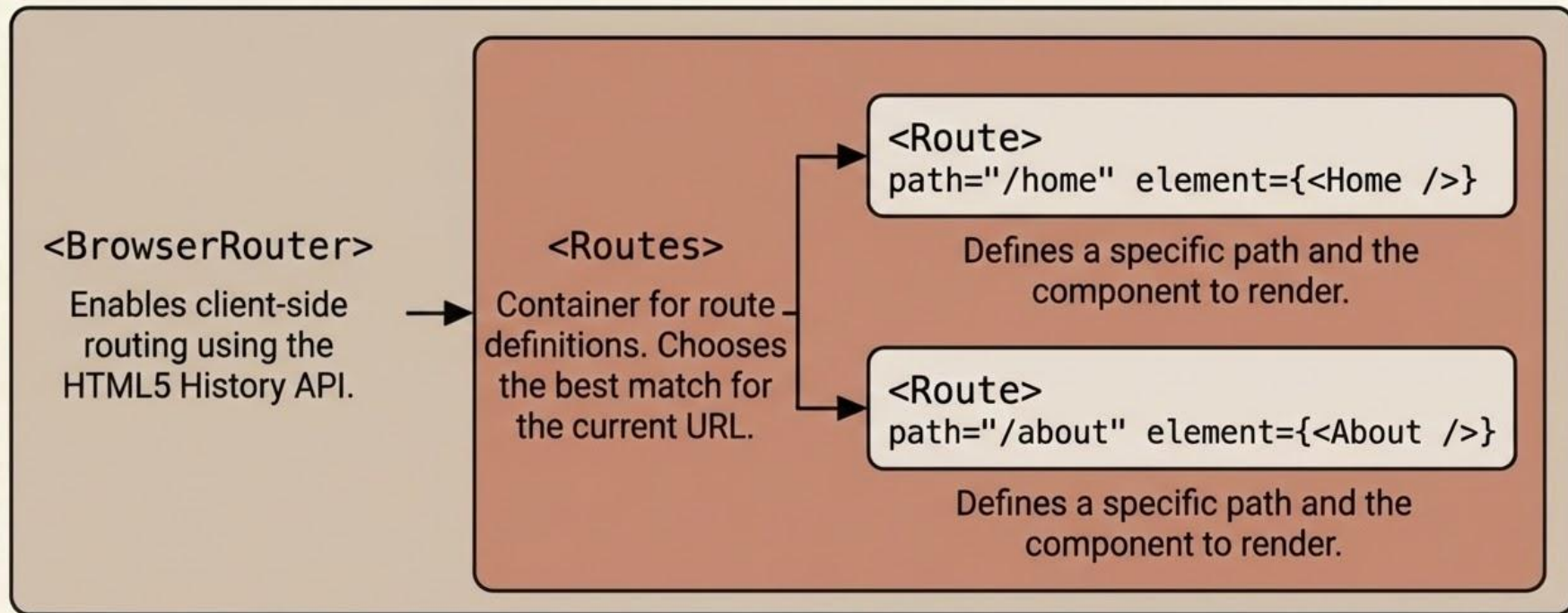
Full Stack Development Subject

```
npm install react-router-dom
```

- Adds the standard routing library for React applications.
- Provides components like `<BrowserRouter>`, `<Route>`, and `<Link>`.
- Enables client-side routing capabilities in your React project.

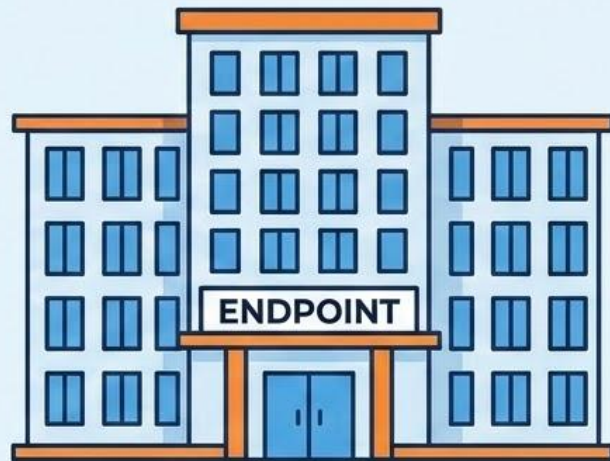
CORE REACT ROUTER COMPONENTS

Full Stack Development Subject



URL & ENDPOINT: THE ADDRESS & THE BUILDING

Real-Life Analogy



URL (Uniform Resource Locator):

The complete address that specifies a unique location on the web.

ENDPOINT: The specific destination or service at that address, like the building itself, ready to receive requests.

ROUTING: THE RECEPTIONIST

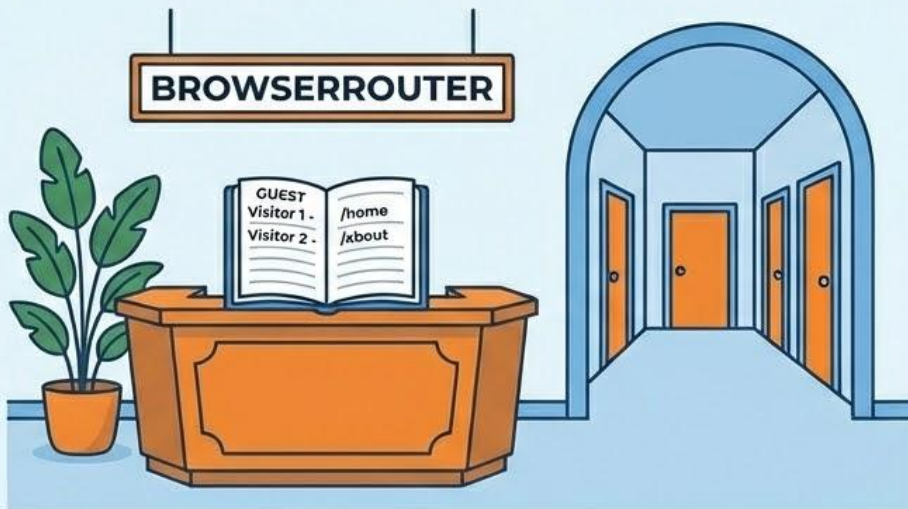
Real-Life Analogy



ROUTING: The process of directing a visitor (the request) from the main entrance to the correct department or room based on their purpose.

BROWSERROUTER: THE LOBBY & LOGBOOK

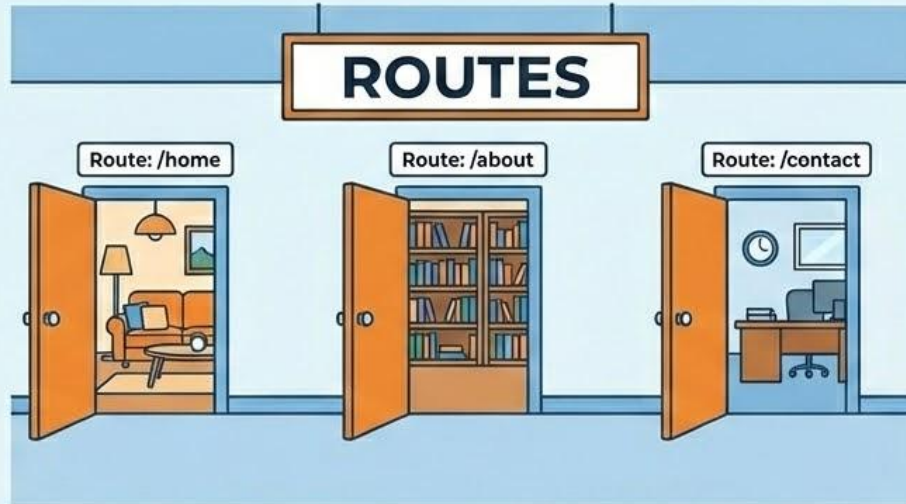
Real-Life Analogy



BROWSERROUTER: The component that manages the application's history (the logbook) and listens for changes in the URL, acting as the main entry point.

ROUTES & ROUTE: THE HALLWAY & THE ROOMS

Real-Life Analogy

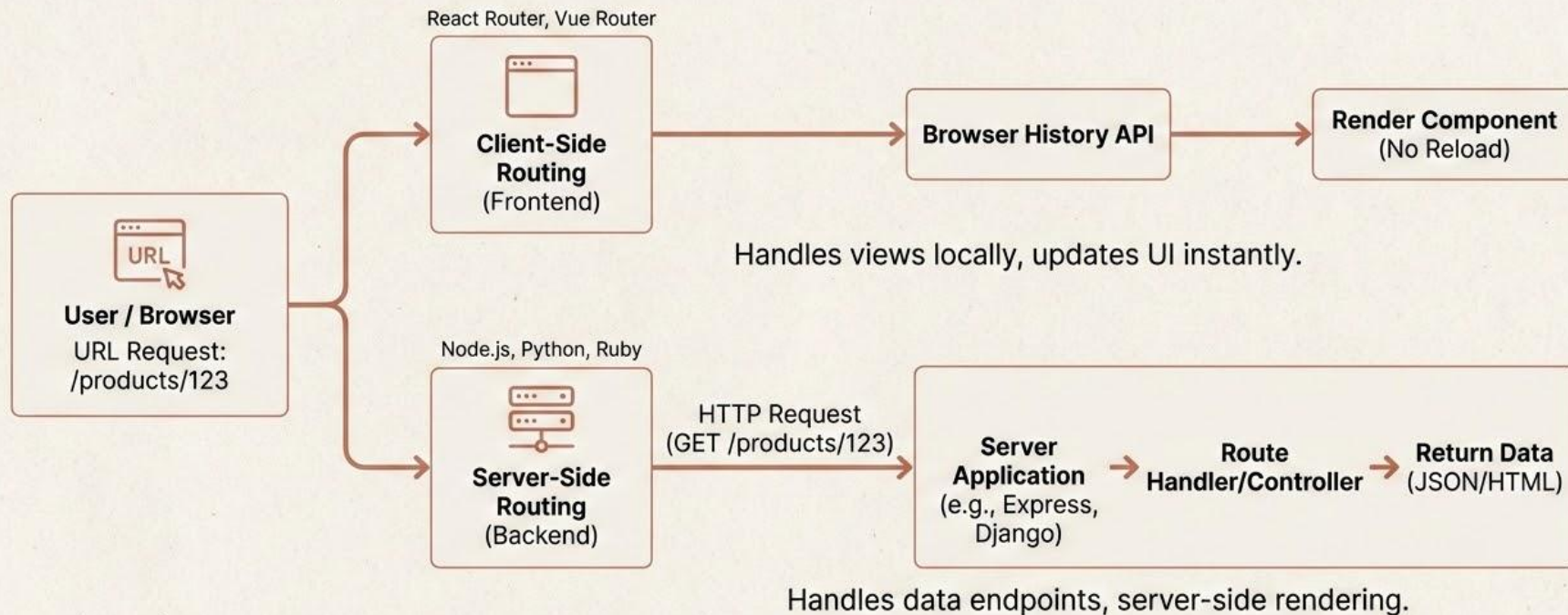


ROUTES: The collection of all possible destinations (the hallway).

ROUTE: A specific door with a path (label) leading to a particular room (component).

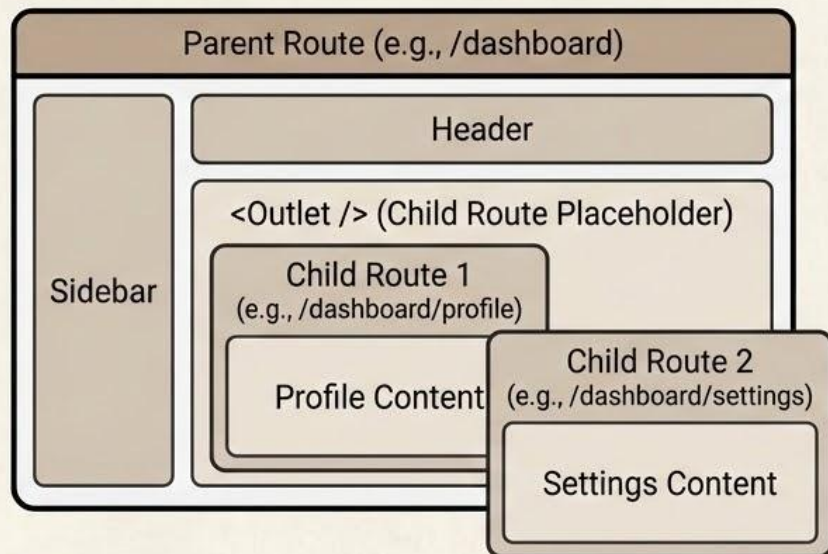
THE CONCEPT OF ROUTING

Full Stack Development Lecture



NESTED ROUTING

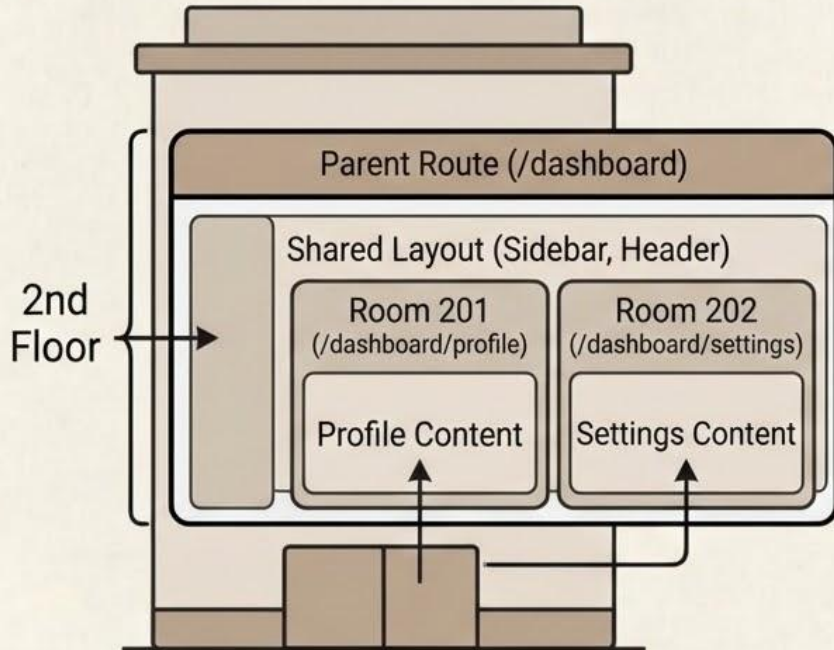
Full Stack Development Subject



- Allows rendering sub-routes within a parent route's component.
- Helps organize complex application UIs into a hierarchy.
- Parent component persists while child components change.
- Great for layouts with shared elements like sidebars or tabs.

MENTAL MODEL: NESTED ROUTING

(Understand This Clearly)



- Think of a building:
- `/dashboard` = 2nd floor
`/dashboard/profile` = Room 201
`/dashboard/settings` = Room 202
- The floor layout (sidebar, header) remains same.
- Only the room content changes.
- That is nested routing.

STEP 1 — ROUTE STRUCTURE

```
<Routes>
  <Route path="/dashboard"
    element={<Dashboard />}>
    <Route path="profile"
      element={<Profile />} />
    <Route path="settings"
      element={<Settings />}/>
  </Route>
</Routes>
```

Important:

- Child routes DO NOT start with /.
- Wrong: path="/profile" ❌
- Correct: path="profile" ✅

STEP 2 — USE OUTLET (MOST IMPORTANT PART)

Inside **Dashboard.jsx**:

```
import { Outlet, Link } from "react-router-dom";

function Dashboard() {
  return (
    <div>
      <h1>Dashboard Layout</h1>
      <nav>
        <Link to="profile">Profile</Link>
        <Link to="settings">Settings</Link>
      </nav>
      <Outlet />
    </div>
  );
}

export default Dashboard;
```

- <Outlet /> is the placeholder.
- It says: **“Render the child route here.”**
- If you forget Outlet → nested routing will **NOT work.**

HOW IT ACTUALLY WORKS INTERNALLY

Flow:

URL = /dashboard/profile



Router matches /dashboard



Router then matches profile inside dashboard



Dashboard renders



Outlet renders Profile component

So rendering is layered. Parent renders first. Child renders inside it.



5 Index Route (Very Important)

What if user visits: /dashboard

But not /dashboard/profile?

You need default child route.

Use:

```
<Route path="/dashboard" element={<Dashboard />}>
  <Route index element={<DashboardHome />} />
  <Route path="profile" element={<Profile />} />
</Route>
```

index means:

- Default child route.
- It renders when path exactly matches parent.

ACCESSING ROUTES & LINKS

How to Navigate Between Routes

Accessing the Parent Route

- Use `<Link>` component. `to` prop specifies the path.

```
<Link to="/dashboard">Go to Dashboard</Link>
```

Note: This links to the parent route.

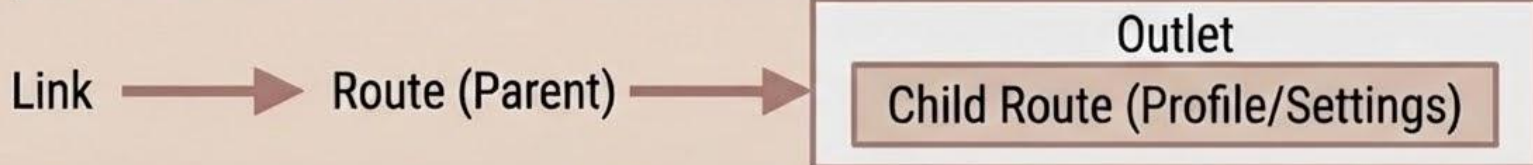
Accessing the Child Route (Relative Links)

- Inside Parent component, use relative paths.

```
<nav>  
  <Link to="profile">Profile</Link>  
  <Link to="settings">Settings</Link>  
</nav>
```

Note: Relative paths ('/dashboard/profile', '/dashboard/settings').

Key Concept: Outlet Renders Child Route Inside Parent





URL PARAMS & DYNAMIC ROUTES

This is not just syntax. This is how dynamic apps work. All concepts are from React Router.



1 What Are URL Params?

URL params are dynamic values inside the URL path.

Examples:

`/product/1`
`/product/25`
`/product/999`

Instead of writing 100 routes like `/product1`, `/product2`, etc. You write **ONE** dynamic route.



3 How To Access It

Inside `Product.jsx`:

```
import { useParams } from "react-router-dom";  
function Product() {  
  const { id } = useParams();  
  return <h1>Product ID: {id}</h1>;  
}
```

If URL is: `/product/42`. Output: **Product ID: 42**



2 Basic Syntax

```
<Route path="/product/:id" element=  
  {<Product />} />
```

The `:id` is a parameter. It means: Accept anything here and store it in a variable called `"id"`.



Mental Model

Think of it like a function.
Route: `/product/:id` is like:
`function product(id) { }`.
URL fills the argument.



4 Why This Is Powerful

Because real apps are dynamic.

Examples:

`/user/ganesh`, `/user/rajesh`, `/user/suresh`
(Instead of creating 3 routes)
`/blog/react-routing`, `/blog/javascript-basics`
(One route handles all)

404 NOT FOUND ROUTE

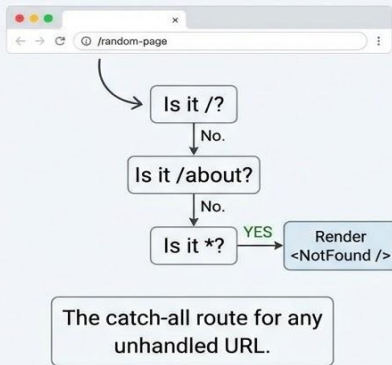
Handling Undefined Routes in React Router

Concept & Syntax

- Use `path="*" element={<NotFound />}` to match any URL that doesn't match other routes.

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="*" element={<NotFound />} />
</Routes>
```

Mental Model



useNavigate HOOK

Programmatic Navigation in React Router

Concept & Usage

- Navigate programmatically, e.g., after a form submission or an action.

```
import { useNavigate } from
'react-router-dom';

function LoginForm() {
  const navigate = useNavigate();

  const handleSubmit = () => {
    // ... login logic ...
    navigate('/dashboard');
  };

  return <button onClick=
    {handleSubmit}>Login</button>;
}
```

Mental Model

